

**UNIVERSIDAD POLITÉCNICA DE
AGUASCALIENTES**
INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN E
INNOVACIÓN DIGITAL



Tarea 03: Aplicación de estándares

(ISO/IEC 25010, IEEE 730, CMMI, MoProSoft)

ASIGNATURA

Estándares y Métricas para el Desarrollo de Software

DOCENTE

Dr(c). Raul Alejandro Velasquez Ortiz

PROYECTO INTEGRADOR

Outline - Wiki interna estilo Notion (Open Source maduro)

EQUIPO No. 4

CHAIRES ALBA OLIVIA LIZBETH - UP240594

ESTRADA RAMÍREZ NOÉ EZEQUIEL - UP240770

FACIO PALOS OMAR - UP240470

JAUREGUI DE LA RIVA ALAN RICARDO - UP240960

RUBIO VIRAMONTES BRIAN ODIN - UP250004

GRUPO

TIID05B

Repositorio Git: <https://github.com/UP240594/outline-sqa-docs>

Aguascalientes, México
18 de mayo de 2026

Índice

1. Introducción	2
1.1. Contexto del proyecto	2
1.2. Objetivos de calidad	2
1.3. Estructura del documento	2
2. Descripción del proyecto integrador	2
2.1. Nombre y tipo	2
2.2. Contexto de uso	3
2.3. Usuarios objetivo	3
2.4. Stack tecnológico	3
2.5. Alcance en historias de usuario	3
2.6. Diagrama de alto nivel (arquitectura)	4
3. Análisis de calidad con ISO/IEC 25010	4
3.1. Priorización de Características de Calidad del Producto	4
3.2. Análisis de Calidad en Uso	5
4. KPIs medibles según ISO/IEC 25023	5
5. Plan SQA preliminar según IEEE 730	7
5.1. §1 Purpose (Propósito y alcance)	7
5.2. §3 Management (Roles y responsabilidades)	7
5.3. §5 Standards, practices, metrics	7
5.4. §7 Test (Estrategia de pruebas)	7
5.5. §8 Problem reporting (SLA por severidad)	8
6. Recomendación de modelo de madurez (CMMI / MoProSoft / ISO 33000)	8
6.1. Análisis comparativo de modelos	8
6.2. Contexto específico	8
6.3. Recomendación: MoProSoft (con elementos de ISO 33000)	8
6.4. Implementación propuesta	9
7. Matriz de riesgos de calidad	9
7.1. Plan de contingencia	10
8. Conclusión integral	11
9. Aportes individuales	12

Introducción

Contexto del proyecto

El presente documento corresponde a la Tarea 03 de la asignatura Estándares y Métricas para el Desarrollo de Software, en la cual se aplican los conceptos teóricos de calidad de software al proyecto integrador del equipo: **Outline**, una aplicación web de tipo wiki interna con funcionalidades similares a Notion, de código abierto y con una comunidad activa.

Outline fue seleccionado del Catálogo de 30 alternativas (Semana 1) por su naturaleza madura (más de 5,000 estrellas en GitHub, licencia MIT) y por ser un caso realista para evaluar atributos de calidad como usabilidad, seguridad y fiabilidad. El proyecto será analizado desde la perspectiva de un equipo de desarrollo que busca adoptar, desplegar y posiblemente contribuir a este software en un entorno corporativo.

Objetivos de calidad

- Aplicar el modelo de calidad de producto ISO/IEC 25010:2023 para priorizar las características más relevantes para Outline.
- Definir KPIs cuantificables basados en ISO/IEC 25023.
- Estructurar un Plan SQA preliminar siguiendo IEEE 730-2014.
- Comparar modelos de madurez y recomendar el más adecuado.
- Identificar riesgos de calidad y proponer mitigaciones.

Estructura del documento

El documento se organiza en seis secciones principales: descripción del proyecto, análisis ISO 25010, KPIs, plan SQA, modelo de madurez y matriz de riesgos. Finaliza con conclusión, aportes individuales y referencias.

Descripción del proyecto integrador

Nombre y tipo

Proyecto: Outline - Wiki interna estilo Notion

Tipo: Web (SaaS con opción self-hosted)

Contexto de uso

Outline está diseñado para equipos que necesitan un repositorio centralizado de conocimiento interno. Ofrece despliegue self-hosted gratuito, integración OAuth y editor colaborativo en tiempo real.

Usuarios objetivo

Rol	Necesidad principal
Editor de contenido	Crear y organizar documentos técnicos.
Lector/Consultor	Buscar información rápidamente.
Administrador	Gestionar permisos e invitaciones.
Administrador de sistemas	Desplegar y mantener la instancia.

Cuadro 1: Usuarios objetivo de Outline

Stack tecnológico

- Frontend: React 18+, TypeScript, MobX, Slate.js
- Backend: Node.js + Express, Sequelize
- Bases de datos: PostgreSQL, Redis
- Infraestructura: Docker Compose, AWS S3/MinIO

Alcance en historias de usuario

1. HU-01: Creación de documento
2. HU-02: Edición colaborativa en tiempo real
3. HU-03: Gestión de permisos y equipos
4. HU-04: Búsqueda y navegación
5. HU-05: Historial de versiones

Diagrama de alto nivel (arquitectura)

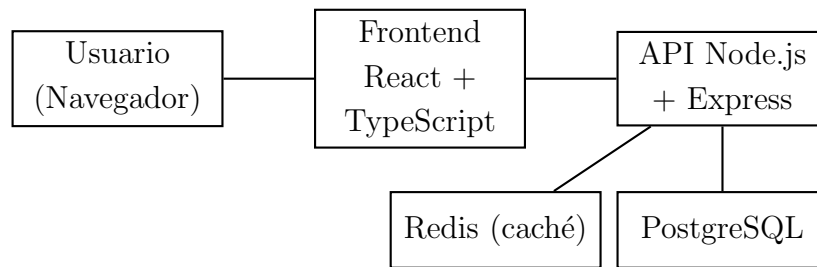


Figura 1: Diagrama de alto nivel de la arquitectura de Outline. Fuente: Elaboración propia.

Análisis de calidad con ISO/IEC 25010

El estándar internacional ISO/IEC 25010:2023 proporciona el marco estructural para diagnosticar y asegurar la calidad del producto de software. En el contexto de **Outline**, concebido como una plataforma de gestión del conocimiento de alto rendimiento y estilo wiki, los atributos de calidad han sido mapeados y priorizados de acuerdo con su impacto crítico en entornos de productividad empresarial distribuidos.

Priorización de Características de Calidad del Producto

- **Adecuación Funcional [Prioridad: ALTA]**
Outline debe garantizar que sus funcionalidades de edición colaborativa en tiempo real, renderizado de Markdown y control de versiones cumplan con exactitud las expectativas del usuario corporativo.
- **Eficiencia de Desempeño [Prioridad: ALTA]**
Los tiempos de carga de documentos extensos y la latencia del motor de búsqueda deben ser mínimos, asegurando un uso fluido.
- **Usabilidad [Prioridad: ALTA]**
La tasa de adopción está ligada a la simplicidad de la experiencia de usuario; una interfaz intuitiva garantiza que personal no técnico colabore sin capacitación compleja.
- **Fiabilidad [Prioridad: ALTA]**
La tolerancia a fallos de sincronización y los mecanismos de respaldo automático son indispensables. La pérdida de documentación interna interrumpe los flujos operativos.
- **Seguridad [Prioridad: ALTA]**
Es obligatorio implementar control de acceso RBAC, soporte SSO/SAML y cifrado de datos en tránsito.

- **Compatibilidad [Prioridad: MEDIA]**

La wiki requiere integración fluida con herramientas externas (Slack, Google Drive, Figma).

- **Mantenibilidad [Prioridad: MEDIA]**

El diseño con NodeJS y React facilita la modularidad. Al ser open source, el equipo debe asegurar legibilidad para integraciones a medida.

- **Portabilidad [Prioridad: MEDIA]**

La instalación se mitiga mediante el uso de contenedores Docker y Kubernetes.

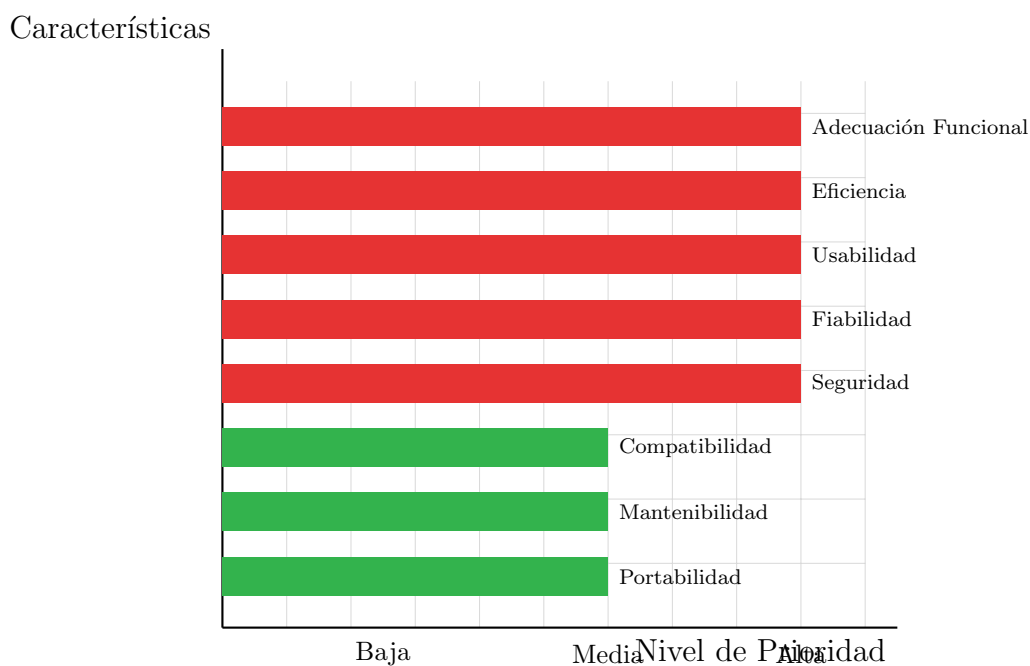


Figura 2: Visualización gráfica del perfil de prioridades de calidad para Outline.

Análisis de Calidad en Uso

Los dos componentes con mayor impacto son:

1. **Efectividad:** Capacidad del software para permitir que los usuarios alcancen metas con precisión. En Outline, se materializa en el guardado síncrono y la precisión del buscador.
2. **Satisfacción:** Respuesta cognitiva y emocional del usuario. El diseño minimalista y los accesos rápidos elevan la satisfacción.

KPIs medibles según ISO/IEC 25023

Con el objetivo de cuantificar objetivamente las características de calidad priorizadas, el equipo ha definido 7 KPIs basados en la norma ISO/IEC, 2016.

Nombre del KPI	Caract. ISO 25010	Fórmula	Umbral	Herramienta
Tasa de éxito de creación de documentos	Adecuación funcional	$\frac{Docscreadosexitosamente}{Intentosdecreación} \times 100$	$\geq 99,5 \%$	Logs de API + Grafana
Disponibilidad del servicio	Fiabilidad	$\frac{T_{up}}{T_{up} + T_{inactivo}}$	$\geq 99,9 \%$	Uptime Kuma
Tiempo de carga del dashboard	Eficiencia (rendimiento)	Percentil 95 (P95) en ms	$< 1,5 \text{ s}$	Lighthouse CI
Tiempo de respuesta de búsqueda	Eficiencia	P95 de latencia del endpoint	$< 500 \text{ ms}$	Prometheus + Grafana
Cobertura de pruebas de seguridad	Seguridad	$\frac{Pruebasejcutadas}{Totalcasos} \times 100$	$\geq 80 \%$	OWASP ZAP + SonarQube
Tasa de retención de usuarios	Calidad en uso	$\frac{Usuariosactivos30d}{Usuariosnuevosmes} \times 100$	$\geq 70 \%$	Analytics propio
Tiempo para primera edición	Usabilidad	Tiempo desde registro hasta primera edición	$< 3 \text{ min}$	Session replay

Cuadro 2: KPIs de Calidad del Producto

Nota metodológica

1. **Línea base:** Establecer medición inicial después del primer sprint Galin, 2018.
2. **Automatización:** KPIs de rendimiento y seguridad deben medirse en cada commit mediante CI/CD Forsgren et al., 2018.
3. **Justificación de umbrales:** Basada en benchmarks de la industria y recomendaciones de Galin.
4. **Percentil P95:** Refleja mejor la experiencia real del usuario que el promedio Pressman y Maxim, 2020.

Plan SQA preliminar según IEEE 730

§1 Purpose (Propósito y alcance)

Plan de Aseguramiento de la Calidad del Software conforme al estándar IEEE 730-2014 IEEE Computer Society, 2014 para el proyecto **Outline**. Establece actividades, responsabilidades y métricas para garantizar el cumplimiento de requisitos funcionales y no funcionales.

Alcance: Aplica a todos los entregables del proyecto Outline: código fuente, documentación técnica, manuales de usuario, informes de pruebas. Cubre las fases de análisis, diseño, implementación, pruebas y despliegue.

§3 Management (Roles y responsabilidades)

Cuadro 3: Roles y responsabilidades en el plan SQA (adaptado al equipo)

Rol	Responsabilidades
Gerente de Proyecto (Noé E. Estrada)	Aprobar el plan SQA, asignar recursos, supervisar hitos de calidad.
Líder de Calidad / QA Tester (Omar Facio)	Ejecutar auditorías internas, medir KPIs, reportar no conformidades.
Desarrolladores (Olivia Chairez, Odin Rubio)	Seguir estándares de codificación, realizar pruebas unitarias.
Documentador (Alan Jauregui)	Mantener actualizada la documentación de calidad.
Cliente / Usuario clave	Validar criterios de aceptación, reportar problemas en fase de pruebas.

§5 Standards, practices, metrics

Estándares: PEP 8 (Python), ESLint (JavaScript), $\text{\LaTeX}$ para documentación, ISTQB v4.0 para pruebas, Git Flow.

Prácticas: Commits con ID de tarea, pull requests con revisión y análisis estático, reuniones semanales con revisión de KPIs.

Métricas: Disponibilidad diaria, cobertura de pruebas por sprint, tiempo de respuesta semanal, etc.

§7 Test (Estrategia de pruebas)

1. Unitarias: JUnit/Pytest, cobertura mínima 80 %.

2. Integración: Postman/Newman en cada CI.
3. Sistema: Selenium, JMeter.
4. Aceptación: Con cliente en preproducción.

Automatización con GitHub Actions.

§8 Problem reporting (SLA por severidad)

Cuadro 4: SLA para reporte de problemas

Severidad	Descripción	Tiempo respuesta	Solución
Crítica (S1)	Pérdida de datos, caída total	2 horas	24 horas
Alta (S2)	Funcionalidad principal afectada	8 horas	3 días hábiles
Media (S3)	Funcionalidad secundaria afectada	2 días hábiles	7 días hábiles
Baja (S4)	Error estético o mejora menor	5 días hábiles	Siguiente sprint

Reporte mediante GitHub Issues.

Recomendación de modelo de madurez (CMMI / MoProSoft / ISO 33000)

Análisis comparativo de modelos

Criterio	CMMI-DEV v2.0	MoProSoft	ISO/IEC 33000
Origen	EE.UU.	México	Internacional
Enfoque	Grandes contratistas	PYMEs mexicanas	Evaluación de capacidad
Niveles	5	3	6
Costo	Alto	Bajo	Medio

Cuadro 5: Comparativa de modelos de madurez

Contexto específico

Equipo de 5 estudiantes, PYME micro, sector software de productividad, mercado México + internacional.

Recomendación: MoProSoft (con elementos de ISO 33000)

- Alineación con tamaño del equipo (diseñado para PYMEs mexicanas) Oktaba y Piattini, 2008.

- Costo documental cero (norma NMX-I-059 pública).
- Estructura simplificada (Alta Dirección, Gerencia, Operación).
- Relevancia para el mercado mexicano.
- Complemento con ISO 33000 para medición de capacidad.

Implementación propuesta

- Alta Dirección: seguimiento de KPIs y asignación de roles.
- Gerencia: gestión de requisitos y planificación.
- Operación: estándares de codificación y estrategia de pruebas.

Matriz de riesgos de calidad

Se identifican 7 riesgos específicos del proyecto Outline.

Riesgo	Característica afectada	Prob.	Impacto	Mitigación
R1: Exposición de secretos en logs	Seguridad	M	A	git-secrets, rotar secretos
R2: Conflictos en editor colaborativo	Fiabilidad	M	A	Pruebas de estrés, backups cada 5 min
R3: Curva de aprendizaje alta	Usabilidad	A	M	Tutorial interactivo, encuestas
R4: Código extenso (50k+ líneas)	Mantenibilidad	A	M	Documentación con Typedoc
R5: Vulnerabilidades en npm	Seguridad	A	A	npm audit semanal, Snyk
R6: Rendimiento en búsquedas	Eficiencia	M	A	Indexar Elasticsearch, pruebas k6
R7: Falta de participación	Adecuación funcional	A	M	Reuniones semanales, checklist en PR

Cuadro 6: Matriz de riesgos de calidad

Leyenda: A = Alta, M = Media, B = Baja

Plan de contingencia

- **R5:** Vulnerabilidad crítica sin parche → bloqueador de release, buscar workaround.
- **R2:** Pérdida de datos → shadow log que registra todas las operaciones.

Conclusión integral

La aplicación de los estándares ISO/IEC 25010, IEEE 730, CMMI y MoProSoft al proyecto Outline ha permitido al equipo descubrir varios aprendizajes clave:

Hallazgo 1: La priorización es esencial. No todas las características de calidad tienen el mismo peso. Para una wiki interna, usabilidad y seguridad dominan sobre portabilidad.

Hallazgo 2: Los KPIs deben ser realistas. Definir umbrales como disponibilidad 99,9 % es alcanzable; 99,999 % no lo es para un proyecto open-source.

Hallazgo 3: IEEE 730 es estructurado pero adaptable. Adaptarlo a un equipo de 5 personas requirió recortar secciones no aplicables.

Hallazgo 4: MoProSoft es relevante y vigente. Sigue siendo pertinente para PY-MEs locales, con principios de baja burocracia.

Conexión con próximos parciales: Los KPIs definidos serán la base para las pruebas del Parcial 2, y la matriz de riesgos alimentará el informe técnico del Parcial 3.

Aportes individuales

Chairez Alba Olivia Lizbeth (UP240594) - Coordinadora: Redacté la portada, introducción, descripción del proyecto (sección 2.1) y conclusión. Unifiqué y armonicé todas las partes de mis compañeros en un solo documento LaTeX, corregí errores de compilación y formateé el documento final. Gestioné el repositorio Git y aseguré la consistencia general. Añadí el diagrama de alto nivel requerido.

Estrada Ramírez Noé Ezequiel (UP240770) - Líder: Desarrollé la sección 2.2 completa (análisis ISO 25010), incluyendo la priorización de las 8 características, justificaciones específicas para Outline y el análisis de calidad en uso (efectividad y satisfacción). Además, elaboré el gráfico de barras con TikZ.

Facio Palos Omar (UP240470) - QA Tester: Elaboré la sección 2.3 (KPIs según ISO 25023), definiendo 7 indicadores con fórmulas, umbrales numéricos y herramientas de medición. Redacté la nota metodológica justificando el uso del percentil P95 y la automatización en CI/CD.

Jauregui De La Riva Alan Ricardo (UP240960) - Documentador: Redacté la sección 2.4 (Plan SQA según IEEE 730), incluyendo propósito, gestión, estándares, estrategia de pruebas y SLA por severidad. Adapté el estándar al contexto del equipo y corregí el nombre del proyecto a Outline.

Rubio Viramontes Brian Odin (UP250004) - Developer: Desarrollé la sección 2.5 (recomendación de modelo de madurez) con análisis comparativo CMMI/MoProSoft/ISO 33000, justificando MoProSoft como la opción óptima. También elaboré la sección 2.6 (matriz de riesgos) con 7 riesgos identificados, probabilidad, impacto y estrategias de mitigación.

Referencias

- Forsgren, N., Humble, J., & Kim, G. (2018). *Accelerate: The Science of Lean Software and DevOps*. IT Revolution.
- Galin, D. (2018). *Software Quality: Concepts and Practice*. Wiley-IEEE Computer Society Press.
- IEEE Computer Society. (2014). *IEEE Standard for Software Quality Assurance Processes*. <https://doi.org/10.1109/IEEESTD.2014.6835311>
- ISO/IEC. (2016). *ISO/IEC 25023:2016 — Measurement of product quality*.
- Oktaba, H., & Piattini, M. (2008). *Software Process Improvement for Small and Medium Enterprises*. IGI Global.
- Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th). McGraw-Hill.